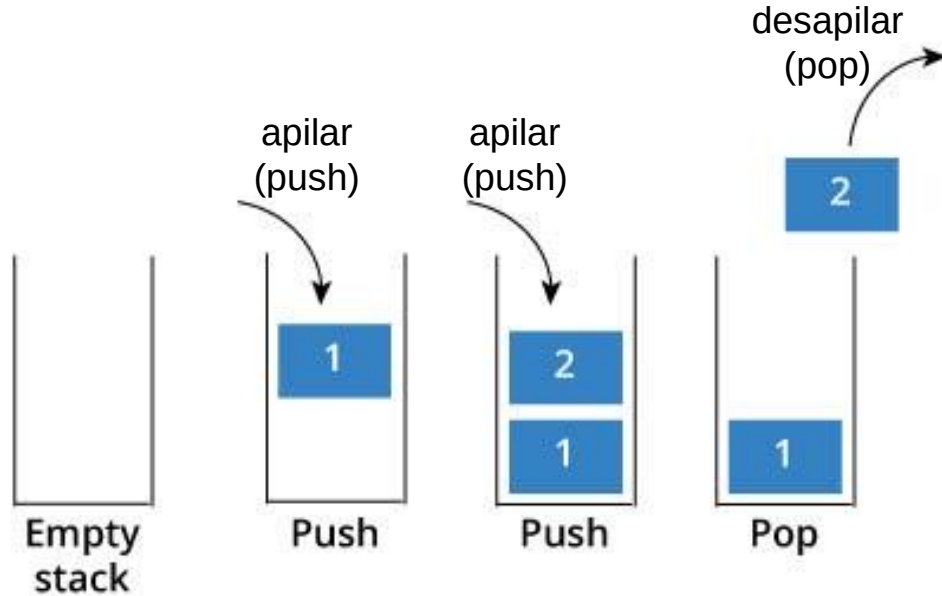


Ensamblador ARM instrucciones

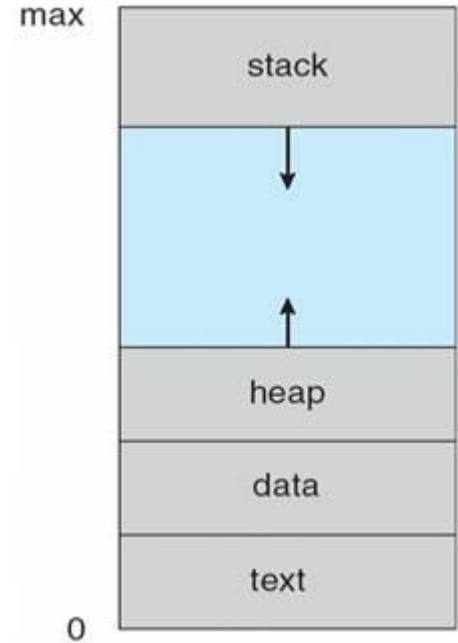
Instrucciones de Push y Pop
Ejemplos ARM

El stack (la pila)



El stack es una estructura de datos LIFO (Last In First Out)

El stack se almacena en un sector de la memoria asignada a mi programa en ejecución



Instrucciones PUSH y POP

PUSH <listreg>

Push guarda/almacena uno ó una lista de registros <listreg> en el stack. El puntero a la pila, es el reg r13, por convencion lo llamamos SP (stack pointer). Este registro siempre apunta a la palabra del tope de la pila, última palabra ingresada en el stack.

POP <listreg>

Pop desapila uno ó una lista de registros <listreg> almacenados en el stack, para acceder a ello usa el registro sp.

Ejemplos

```
mov r1, #123    //r1=123  
push r1        //guardo el registro r1 en la pila  
....          // código que puede usar el registro r1  
mov r1, #255    //r1=255  
....  
pop r1  //restaurar el registro r1 de la pila, r1=123
```

Registro SP - Stack Pointer (R13)

Registers azeria-labs.com

R0	R1	R2
0x00000002	0x00000000	0x00000000

```
_start:  
    mov    r0, #2  
    push   {r0}  
    mov    r0, #3  
    pop    {r0}
```

Diagram illustrating the stack structure and the Stack Pointer (SP) register:

full descending

Registro SP - Stack Pointer (R13)

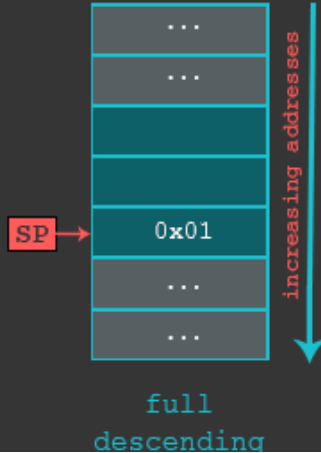
Registers

azeria-labs.com

R0	R1	R2
0x00000002	0x00000000	0x00000000

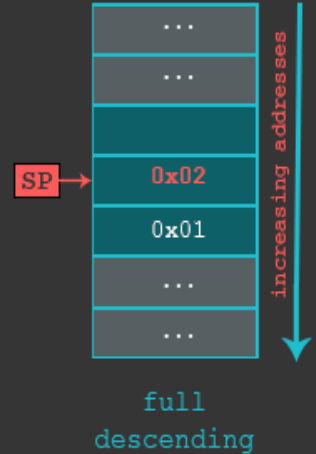
_start:

```
mov  r0, #2
push {r0}
mov  r0, #3
pop  {r0}
```



_start:

```
mov  r0, #2
push {r0}
mov  r0, #3
pop  {r0}
```



Registro SP - Stack Pointer (R13)

Registers

azeria-labs.com

R0	R1	R2
0x00000003	0x00000000	0x00000000

_start:

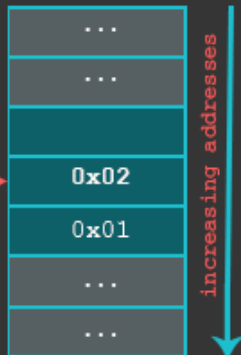
```
mov r0, #2
```

```
push {r0}
```

```
mov r0, #3
```

```
pop {r0}
```

SP →



full
descending

Registers

azeria-labs.com

R0	R1	R2
0x00000002	0x00000000	0x00000000

_start:

```
mov r0, #2
```

```
push {r0}
```

```
mov r0, #3
```

```
pop {r0}
```

SP →



full
descending

Orden del PUSH y POP

Por la naturaleza del stack, se desapila en orden inverso en que se apilaron

Ejemplos

```
push r1
```

```
push r2
```

```
push r4
```

```
// código que modifica los registro r1, r2 y r4
```

```
pop r4
```

```
pop r2
```

```
pop r1
```

Si no cumpla este orden, puedo intercambiar los valores entre los registros, en lugar de solamente guardarlos y restaurarlos

Swap con PUSH y POP

Suponer que r0=10 y r1=50

Hacemos el swap con registros:



//r0 = 10

// r1 = 50

mov r2, r1

mov r1, r0 // r1 =10

mov r0, r2 // r0 =50

Swap con PUSH y POP

Suponer que $r0=10$ y $r1=50$

Problema: Intercambiar sus valores sin usar ningún otro registro adicional

Pista: usamos la pila!



```
mov r0, 10
```

```
mov r1, 50
```

```
/*ahora hacemos el swap*/
```

```
push r1
```

```
mov r1, r0 /*r1=10*/
```

```
pop r0 /*estamos desapilando en  
el registro r0, es decir, copiamos  
el tope de la pila al registro r0,  
r0=50*/
```

Ejercicio

Suponer que tiene dos datos en memoria:

.data

var1 : .word 0xFFFFFFFF

var2 : .word 0x0

Hacer un programa que hace un swap entre var1 y var2 **usando la pila**.

Verificar con gdb que el swap se realizó, es decir, la memoria debe quedar con los siguientes valores

var1 : .word 0x0

var2 : .word 0xFFFFFFFF

Ayuda, para ver el contenido de la memoria con
gdb es: x/xw &var1 (corregir esto)